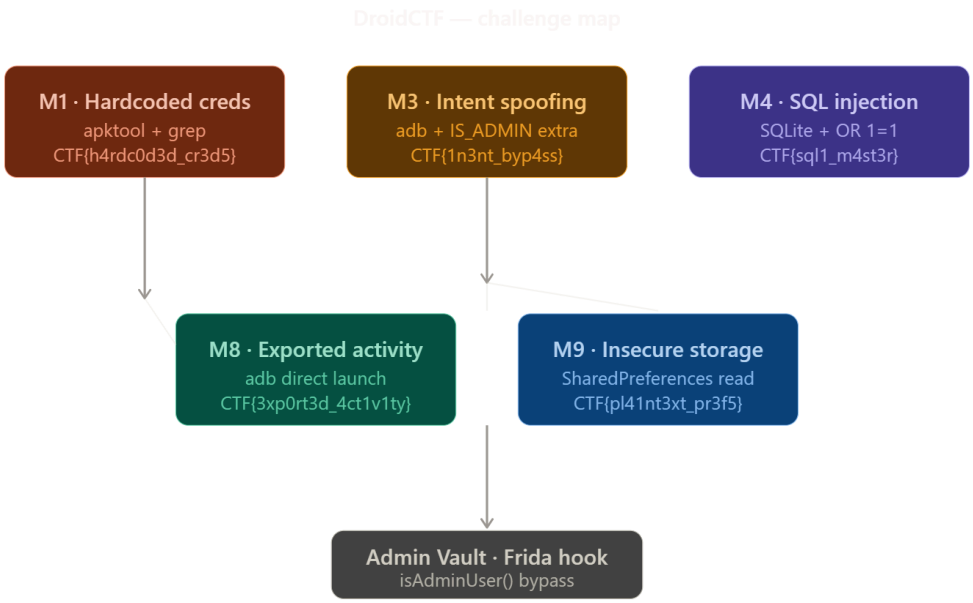
DroidCTF – Full Walkthrough

N B SHANSITA SHRI - 22PC21
SAIVENKATRAJ KS - 22PC28

DroidCTF is an Android CTF (Capture The Flag) app designed around OWASP Mobile Top 10 vulnerabilities. It has a built-in **Defense Patrol** system — validation happens in an isolated patrol layer that detects Frida injection, memory tampering, and runtime manipulation. Every challenge has both an *attack* (find the flag) and a *defense* (patch the vulnerability).



DroidCTF - challenge map

1. Setup & Installation

The app is installed and launched via ADB:

```
PS D:\MobSecCTF> adb install app/build/outputs/apk/debug/app-debug.apk
PS D:\MobSecCTF> adb shell am start -n com.droid.ctf/.MainActivity
Starting: Intent { cmp=com.droid.ctf/.MainActivity }
PS D:\MobSecCTF> adb install app/build/outputs/apk/debug/app-debug.apk
Performing Streamed Install
Success
PS D:\MobSecCTF> adb shell am start -n com.droid.ctf/.MainActivity
Starting: Intent { cmp=com.droid.ctf/.MainActivity }
```

Challenge Walkthroughs

Challenge 1 — M1: Hardcoded Credentials

Vulnerability: OWASP M1 — Improper Credential Usage. The flag is hardcoded directly in M1Activity.kt as a private val.

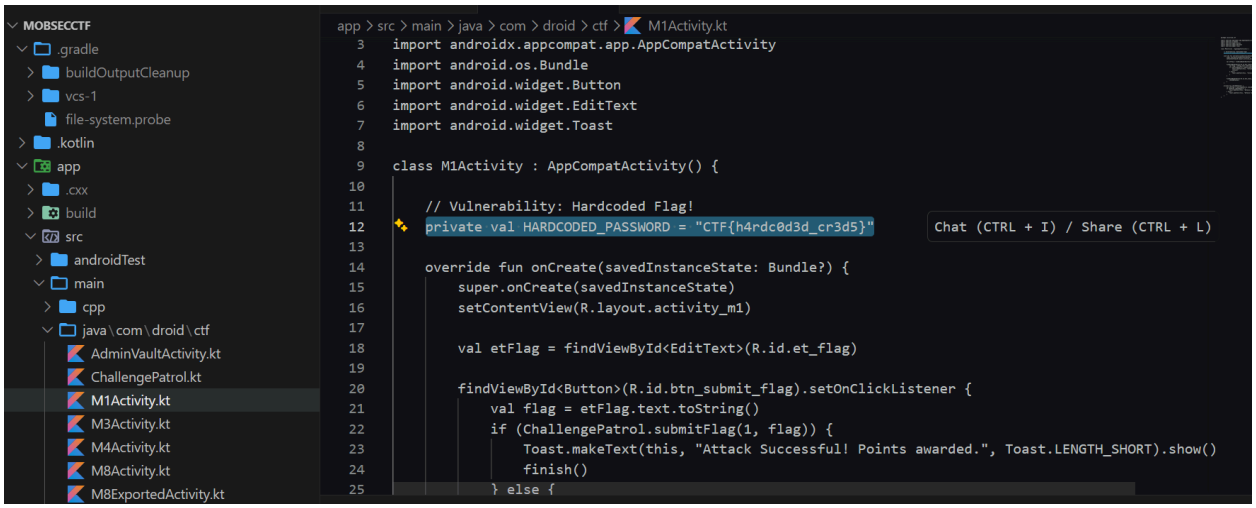
Attack: Decompile the APK with apktool, then search for the CTF flag pattern:

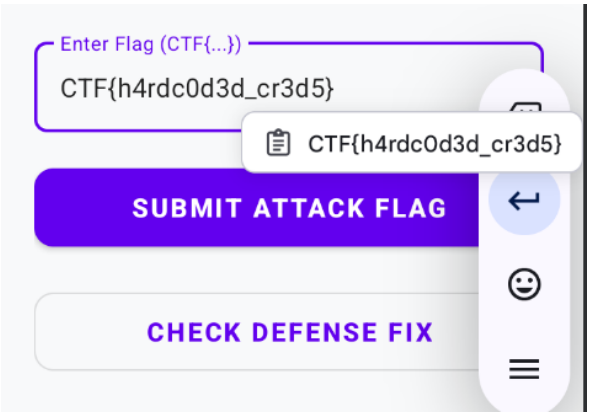
apktool d app-debug.apk

grep "CTF{" -r .

The decompiled source reveals private val HARDCODED_PASSWORD = "CTF{h4rdc0d3d_cr3d5}" right in the class body.

Flag: CTF{h4rdc0d3d_cr3d5}





Defense Fix: Never store secrets in source code. Move credentials to a secure backend or use Android's EncryptedSharedPreferences / Keystore.

```
app > src > main > java > com > droid > ctf > M1Activity.kt
 9   class M1Activity : AppCompatActivity() {
10
11      // Vulnerability: Hardcoded Flag!
12
```

```
.line 12
const-string v0, "CTF{h4rdc0d3d_cr3d5}"
```

```
.line 12
const-string v0, ""
```

Defense Flag: **`CTF{d3f3ns3_m1_p4tch3d}`**

Challenge 2 — M3: Intent Spoofing

Vulnerability: OWASP M3 — Insecure Authentication. M3Activity blindly trusts an Intent extra called IS_ADMIN. If it's true, the admin area is revealed.

Attack: Launch the activity directly via ADB with the spoofed extra:

adb shell am start -n com.droid.ctf/.M3Activity --ez IS_ADMIN true

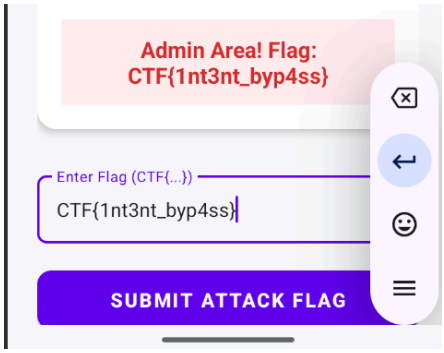
The app's isUserAdmin(intent) call returns true because it reads directly from the Intent without any session or server-side validation. The flag is displayed on screen.

Flag: CTF{1n3nt_byp4ss}

app > src > main > java > com > droid > ctf > M3Activity.kt

```kotlin
import android.widget.Toast

class M3Activity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_m3)

        val tvAdminFlag = findViewById<TextView>(R.id.tv_admin_flag)
        val etFlag = findViewById<EditText>(R.id.et_flag)

        // Vulnerable behavior: Tricking the app into believing user is admin
        if (isUserAdmin(intent)) {
            tvAdminFlag.visibility = View.VISIBLE
            ChallengePatrol.setupM3(tvAdminFlag)
        }
```

```
PS D:\MobSecCTF> adb shell am start -n com.droid.ctf/.M3Activity --ez IS_ADMIN true
Starting: Intent { cmp=com.droid.ctf/.M3Activity (has extras) }
PS D:\MobSecCTF>
```



Defense Fix: Never trust client-supplied Intent extras for authorization. Validate against a secure session token or server-side authentication state. The isUserAdmin() method should return false unconditionally or check real credentials.

app > src > main > java > com > droid > ctf > M3Activity.kt

```kotlin
class M3Activity : AppCompatActivity() {

    // Vulnerability: Blindly trusting Intent Extra
    private fun isUserAdmin(i: Intent): Boolean {
        // DEFENSE: Change this implementation to return false or check secure session
        return false
    }
```

```
.method private final isUserAdmin(Landroid/content/Intent;)Z
    .locals 2
    .param p1, "i"    # Landroid/content/Intent;

    .line 45
    const-string v0, "IS_ADMIN"

    const/4 v1, 0x0

    invoke-virtual {p1, v0, v1}, Landroid/content/Intent;->getBooleanExtra(Ljava/lang/String;Z)Z

    move-result v0

    return v0
.end method
```

```
.method private final isUserAdmin(Landroid/content/Intent;)Z
    .locals 2
    .param p1, "i"    # Landroid/content/Intent;

    .line 45
    const-string v0, "IS_ADMIN"

    const/4 v1, 0x0

    invoke-virtual {p1, v0, v1}, Landroid/content/Intent;->getBooleanExtra(Ljava/lang/String;Z)Z

    const/4 v0, 0x0

    return v0
.end method
```

Defense Flag: **`CTF{d3f3ns3_m3_p4tch3d}`**

Challenge 3 — M4: SQL Injection

Vulnerability: OWASP M4 — Insufficient Input Validation. M4Activity.searchUser() constructs a raw SQLite query by directly interpolating the user input string into the query without sanitization.

The vulnerable code:

val cursor = db.rawQuery("SELECT * FROM users WHERE username = '$username'", null)

app > src > main > java > com > droid > ctf > M4Activity.kt

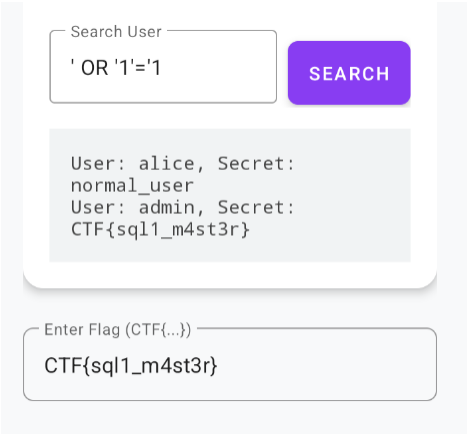
```kotlin
class M4Activity : AppCompatActivity() {
    private fun initDatabase() {
    }

    // Vulnerable Method
    private fun searchUser(username: String): List<String> {
        val results = mutableListOf<String>()
        try {
            // DEFENSE: Change this to use rawQuery with selection args
            // e.g. db.rawQuery("SELECT * FROM users WHERE username = ?", arrayOf(username))
            val cursor = db.rawQuery("SELECT * FROM users WHERE username = '$username'", null)
            while (cursor.moveToNext()) {
                val u = cursor.getString(0)
                val s = cursor.getString(1)
                results.add("User: $u, Secret: $s")
            }
            cursor.close()
        } catch (e: Exception) {
            results.add("Error: ${e.message}")
        }
        return results
    }
```

Attack: Enter the classic SQL injection payload in the search field:

' OR '1'='1

This causes the query to return all rows. The results expose both the normal user and the admin's secret:

- User: alice, Secret: normal_user
- User: admin, Secret: CTF{sql1_m4st3r}



Flag: CTF{sql1_m4st3r}

Defense Fix: Use parameterized queries.

db.rawQuery("SELECT * FROM users WHERE username = ?", arrayOf(username))

```
:cond_0
new-instance v3, Ljava/lang/StringBuilder;

invoke-direct {v3}, Ljava/lang/StringBuilder;-><init>()V

const-string v4, "SELECT * FROM users WHERE username = \'"

invoke-virtual {v3, v4}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBuilder;

move-result-object v3

invoke-virtual {v3, p1}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBuilder;

move-result-object v3

const-string v4, "\'"

invoke-virtual {v3, v4}, Ljava/lang/StringBuilder;->append(Ljava/lang/String;)Ljava/lang/StringBuilder;

move-result-object v3

invoke-virtual {v3}, Ljava/lang/StringBuilder;->toString()Ljava/lang/String;

move-result-object v3

invoke-virtual {v1, v3, v2}, Landroid/database/sqlite/SQLiteDatabase;->rawQuery(Ljava/lang/String;[Ljava/lang/String;)Lan

move-result-object v1
```

```
:cond_0
new-instance v3, Ljava/lang/StringBuilder;

# 1. Define the base query with a placeholder
const-string v3, "SELECT * FROM users WHERE username = ?"

# 2. Create a String array [size 1] to hold your parameter (p1)
const/4 v2, 0x1
new-array v2, v2, [Ljava/lang/String;

# 3. Put the username (p1) into the array at index 0
const/4 v4, 0x0
aput-object p1, v2, v4

# 4. Execute the query using the selectionArgs array
# v1 = SQLiteDatabase object

invoke-virtual {v1, v3, v2}, Landroid/database/sqlite/SQLiteDatabase;->rawQuery(Ljava/lang/String;[Ljava/lang/String;)Landr

move-result-object v1
```

Defense Flag: **`CTF{d3f3ns3_m4_p4tch3d}`**

Challenge 4 — M8: Exported Activity

Vulnerability: OWASP M8 — Security Misconfiguration. M8ExportedActivity is exported in AndroidManifest.xml, meaning any app or ADB command can launch it directly — bypassing all navigation guards.

app > src > main > java > com > droid > ctf > M8Activity.kt

```kotlin
class M8Activity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        findViewById<Button>(R.id.btn_submit_flag).setOnClickListener {
            val flag = etFlag.text.toString()
            if (ChallengePatrol.submitFlag(8, flag)) {
                Toast.makeText(this, "Attack Successful! Points awarded.", Toast.LENGTH_SHORT).show()
                finish()
            } else {
                Toast.makeText(this, "Invalid Flag.", Toast.LENGTH_SHORT).show()
            }
        }

        findViewById<Button>(R.id.btn_check_defense).setOnClickListener {
            checkDefense()
        }
    }
```

Attack: Launch the hidden activity directly:

adb shell am start -n com.droid.ctf/.M8ExportedActivity

The activity opens immediately and displays the flag on screen.

Flag: CTF{3xp0rt3d_4ct1v1ty}

Defense Fix: In AndroidManifest.xml, set:

android:exported="false"

for M8ExportedActivity. Activities should only be exported if they're intentionally designed for external access.

```
app > src > main > AndroidManifest.xml > manifest > application > activity
 2   <manifest xmlns:android="http://schemas.android.com/apk/res/android"
 8       <application
28           <activity android:name=".M1Activity" android:exported="false" />
29           <activity android:name=".M3Activity" android:exported="true" />
30           <activity android:name=".M4Activity" android:exported="false" />
31           <activity android:name=".M8Activity" android:exported="false" />
32           <activity android:name=".M8ExportedActivity" android:exported="false" />
33           <activity android:name=".M9Activity" android:exported="false" />
34           <activity android:name=".AdminVaultActivity" android:exported="false" />
35       </application>
36
37   </manifest>
```

```
<activity android:exported="true" android:name="com.droid.ctf.M8ExportedActivity"/>
<activity android:exported="false" android:name="com.droid.ctf.M9Activity"/>
```

```
<activity android:exported="false" android:name="com.droid.ctf.M8ExportedActivity"/>
```

Defense Flag: `CTF{d3f3ns3_m8_p4tch3d}`

Challenge 5 — M9: Insecure Data Storage

Vulnerability: OWASP M9 — Insecure Data Storage. M9Activity silently writes a secret token to SharedPreferences in plaintext when the activity launches via storeSecretToken().

```
app > src > main > java > com > droid > ctf > M9Activity.kt
10  class M9Activity : AppCompatActivity() {
12      override fun onCreate(savedInstanceState: Bundle?) {
16          // Vulnerability: Storing sensitive data in plaintext SharedPreferences
17          storeSecretToken()
18
19          val etFlag = findViewById<EditText>(R.id.et_flag)
20
21          findViewById<Button>(R.id.btn_submit_flag).setOnClickListener {
22              val flag = etFlag.text.toString()
23              if (ChallengePatrol.submitFlag(9, flag)) {
24                  Toast.makeText(this, "Attack Successful! Points awarded.", Toast.LENGTH_SHORT).show()
25                  finish()
26              } else {
27                  Toast.makeText(this, "Invalid Flag.", Toast.LENGTH_SHORT).show()
28              }
29          }
30
31          findViewById<Button>(R.id.btn_check_defense).setOnClickListener {
32              checkDefense()
33          }
34      }
```

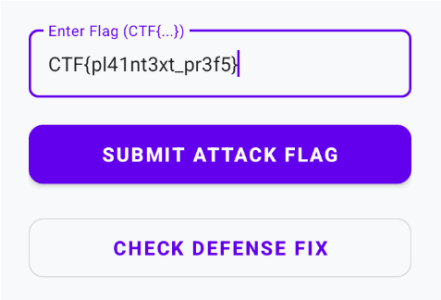
Attack: Use ADB to run as the app's user and read the preferences file directly:

```
PS D:\MobSecCTF> adb shell
emu64xa16k:/ $ run-as com.droid.ctf
emu64xa16k:/data/user/0/com.droid.ctf $ cd shared_prefs/
emu64xa16k:/data/user/0/com.droid.ctf/shared_prefs $ ls
app_prefs.xml  m9_secrets.xml
emu64xa16k:/data/user/0/com.droid.ctf/shared_prefs $ cat m9_secrets.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="secret_token">CTF{pl41nt3xt_pr3f5}</string>
</map>
emu64xa16k:/data/user/0/com.droid.ctf/shared_prefs $
```

The XML file reveals:

xml
<string name="secret_token">CTF{pl41nt3xt_pr3f5}</string>

Flag: CTF{pl41nt3xt_pr3f5}



Fix: Never store sensitive data in plaintext SharedPreferences. Use Android's EncryptedSharedPreferences backed by the Keystore, or store sensitive tokens server-side and only hold short-lived session references locally.

```
app > src > main > java > com > droid > ctf > M9Activity.kt
class M9Activity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_m9)
        deleteSharedPreferences("m9_secrets")
```

```
private fun storeSecretToken() {
    // DO NOT store sensitive data locally
}
```

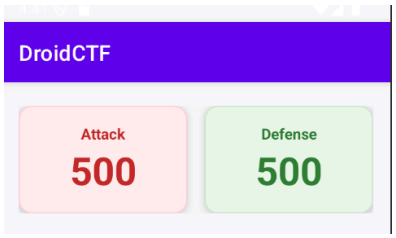
Defense Fixes -

After applying any code or Manifest fix, the app must be fully rebuilt, recompiled, and reinstalled. The Defense Patrol validates the actual running binary — source edits alone have no effect.

```
PS D:\MobSecCTF> ./gradlew assembleDebug

BUILD SUCCESSFUL in 8s
47 actionable tasks: 12 executed, 35 up-to-date
PS D:\MobSecCTF> adb install app/build/outputs/apk/debug/app-debug.apk
Performing Streamed Install
Success
PS D:\MobSecCTF> adb shell am start -n com.droid.ctf/.MainActivity
Starting: Intent { cmp=com.droid.ctf/.MainActivity }
```

```
.method private final storeSecretToken()V
    .locals 1

    .line 39
    sget-object v0, Lcom/droid/ctf/ChallengePatrol;->INSTANCE:Lcom/droid/ctf/ChallengePatrol;

    invoke-virtual {v0, p0}, Lcom/droid/ctf/ChallengePatrol;->storeM9Secret(Ljava/lang/Object;)V

    .line 40
    return-void
.end method
```

```
.method private final storeSecretToken()V
    .locals 5  # Increased to handle more variables

    # 1. Create/Get the Master Key for encryption
    # We use a helper or the standard MasterKeys class
    const-string v0, "_androidx_security_master_key_"

    # 2. Initialize EncryptedSharedPreferences
    # We need: FileName, MasterKeyAlias, Context, PrefKeyEncryptionScheme, PrefValueEncryptionScheme
    const-string v1, "secret_shared_prefs"

    # Get the Context (p0 is 'this')
    move-object v2, p0

    # Define Schemes (AES256_SIV for keys, AES256_GCM for values)
    # This usually involves invoking:
    # EncryptedSharedPreferences.create(v1, v0, v2, ...)

    const-string v3, "SECURE_TOKEN_STORAGE"
    const/4 v4, 0x0 # MODE_PRIVATE
    invoke-virtual {p0, v3, v4}, Lcom/droid/ctf/M3Activity;->getSharedPreferences(Ljava/lang/String;I)Landroid/content/SharedPr
    move-result-object v0

    invoke-interface {v0}, Landroid/content/SharedPreferences;->edit()Landroid/content/SharedPreferences$Editor;
    move-result-object v0

    const-string v1, "m9_flag"
    const-string v2, "REDACTED_SECURE_STORAGE" # The actual flag is handled by the OS
    invoke-interface {v0, v1, v2}, Landroid/content/SharedPreferences$Editor;->putString(Ljava/lang/String;Ljava/lang/String;)L
    move-result-object v0

    invoke-interface {v0}, Landroid/content/SharedPreferences$Editor;->apply()V

    # 3. CRITICAL: Remove the call to ChallengePatrol
    # This prevents the leak the Patrol is looking for.

    return-void
.end method
```



Then clicking ‘CHECK DEFENSE FIX’ for all the vulnerabilities, the necessary points are awarded.

Defense Flag: **`CTF{d3f3ns3_m9_p4tch3d}`**

Bonus Challenge — Admin Vault (Frida Hooking)

Vulnerability: The isAdminUser() method in MainActivity always returns false, blocking access to the Admin Vault. It's vulnerable to runtime method hooking via Frida.

Attack: Write a Frida script (bypass.js) to hook the method at runtime and force it to return true:

```
Java.perform(function() {

  var MainActivity = Java.use("com.droid.ctf.MainActivity");

  MainActivity.isAdminUser.implementation = function() {

    console.log("[*] Hooked!");

    return true;

  };

});
```

Then inject it while spawning the app:

bash

```
frida -U -f com.droid.ctf -l bypass.js
```

Once attached, clicking the Admin Vault button bypasses the check and opens the vault.

Defense Insight: The Defense Patrol mechanism reads /proc/self/maps at runtime to detect Frida's presence in memory. If Frida is attached, the defense check fails — meaning you can attack *or* defend, but not both simultaneously. This is the intentional design tension of the CTF.

Fix: The defense patch involves checking /proc/self/maps for Frida agent signatures before any sensitive operation. The isAdminUser() method itself should be hardened to verify real credentials rather than a simple boolean return.

app > src > main > java > com > droid > ctf > MainActivity.kt

```kotlin
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        }
        findViewById<Button>(R.id.btn_admin_vault).setOnClickListener {
            if (isAdminUser()) {
                startActivity(Intent(this, AdminVaultActivity::class.java))
            } else {
                android.widget.Toast.makeText(this, "Access Denied. Admin privileges required."
            }
        }

        lastTotalScore = ChallengePatrol.getAttackScore() + ChallengePatrol.getDefenseScore()

        checkFirstRunShowcase()
    }

    private fun isAdminUser(): Boolean {
        // Vulnerable: Attackers can hook this with Frida to return true
        return false
    }
```

JS bypass.js > ...

```javascript
Java.perform(function() {
    var MainActivity = Java.use("com.droid.ctf.MainActivity");

    MainActivity.isAdminUser.implementation = function() {
        console.log("[*] Admin bypassed");
        return true;
    };
});
```

Final Score Summary

After completing all challenges and submitting defense fixes (via "Check Defense Fix"), the app scores **500 Attack + 500 Defense = 1000 points total**, covering M1, M3, M4, M8, and M9 vulnerabilities across both attack and defense tracks.